Fundamental 1팀 스터디 / 딥 러닝 파이토치 교과서

17
어텐션 메커니즘

16장의 번역기는 입력 문장 전체를 벡터 하나에 눌러 담았습니다. 어텐션은 디코더가 단어를 만들 때마다 입력 문장의 모든 단어를 다시 보고, 지금 필요한 단어에 집중하게 합니다. 트랜스포머의 핵심 부품이 여기서 나옵니다.

오늘 다루는 것

1. 어텐션 메커니즘: 닷-프로덕트 어텐션
2. 바다나우 어텐션
3. 어텐션 메커니즘을 이용한 번역기 구현

끝나고 나면 할 수 있는 것

1. 어텐션 스코어, 어텐션 분포, 어텐션 값을 숫자 예로 직접 계산할 수 있다.
2. Query, Key, Value가 seq2seq에서 각각 무엇인지 말할 수 있다.
3. 어텐션을 더한 번역기가 16장의 번역기와 무엇이 달라졌는지 코드와 결과로 설명할 수 있다.

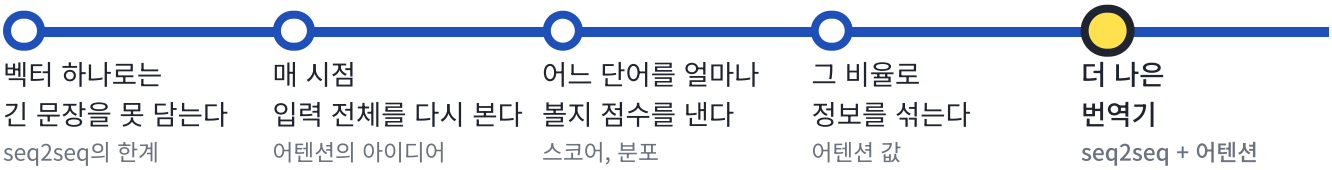
2026년 10월 14일 수요일 14:30 (10/14 자료 2/2)
16장 자료를 읽은 뒤에 봅니다. 실습은 모두 코랩 기본 환경에서 실행됩니다. 실습 3 → 4 순서로 실행합니다(실습 4는 CPU로 약 20분).

17

어텐션 메커니즘 (Attention Mechanism)

긴 글을 번역할 때 사람은 원문을 한 번 읽고 덮어 두지 않습니다. 한 구절을 옮길 때마다 원문의 해당 부분을 다시 봅니다. 어텐션은 번역 모델에게 그 일을 시킵니다. 단어를 하나 만들 때마다 입력 문장 전체를 다시 보되, 지금 필요한 단어를 더 많이 봅니다.

왜 배우나: 오늘의 학습 경로



상황은 16장과 같습니다. 영어 "I am a student"를 프랑스어 "je suis étudiant"로 번역합니다. 17-01에서는 디코더가 세 번째 단어 étudiant를 만드는 순간을 숫자로 따라가고, 17-03에서는 16장과 같은 데이터로 번역기를 다시 학습시켜 비교합니다.

17-01 어텐션 메커니즘

핵심 한 줄

어텐션은 디코더의 현재 은닉 상태(Query)로 인코더의 모든 은닉 상태(Key)와 점수를 내고, 소프트맥스로 비율을 정한 뒤, 그 비율로 인코더의 은닉 상태(Value)를 가중합해 예측에 쓰는 방법이다.

왜 배우나

16장의 seq2seq에는 두 가지 문제가 있었습니다. 입력 문장을 고정된 크기의 벡터 하나에 담느라 정보가 빠지고, RNN이 길어질수록 앞의 내용이 흐려집니다(기울기 소실). 그래서 문장이 길면 번역이 나빠집니다. 어텐션은 이 문제를 풀려고 나왔고, 나중에는 RNN을 대신하게 됩니다(18장). 계산은 여섯 단계이고, 그중 앞의 세 단계가 핵심입니다.

1) 어텐션의 아이디어와 Query, Key, Value

정의

- 어텐션(attention): 디코더가 출력 단어를 예측하는 **매 시점마다** 인코더의 입력 문장 전체를 다시 참고하는 방법. 모든 단어를 같은 비중으로 보지 않고, 지금 예측할 단어와 관련 있는 단어를 더 집중해서 봅니다.
- 어텐션 함수: $\text{Attention}(Q, K, V) = \text{attention value}$. 주어진 쿼리(Query)에 대해 모든 키(Key)와의 유사도를 구하고, 그 유사도를 비율로 삼아 키마다 짝지어진 값(Value)을 섞어 돌려줍니다.

파이썬의 딕셔너리 `{"2017": "Transformer", "2018": "BERT"}` 는 키가 정확히 같을 때만 값을 꺼냅니다. 어텐션은 "키와 얼마나 비슷한가"에 따라 **모든 값을 조금씩** 꺼내 섞는, 부드러운 딕셔너리입니다.

역할	seq2seq + 어텐션에서는	기호
Query (질문)	디코더의 현재 시점 t의 은닉 상태	s_t
Keys (맞춰 볼 대상)	인코더의 모든 시점의 은닉 상태	$h_1, \dots, h_N$
Values (가져올 내용)	인코더의 모든 시점의 은닉 상태 (Key와 같음)	$h_1, \dots, h_N$

N은 입력 문장의 단어 수입니다. 16장에서는 인코더의 마지막 은닉 상태 h_N만 썼습니다. 어텐션은 버리던 $h_1, \dots, h_{N-1}$을 모두 씁니다.

예시 적용

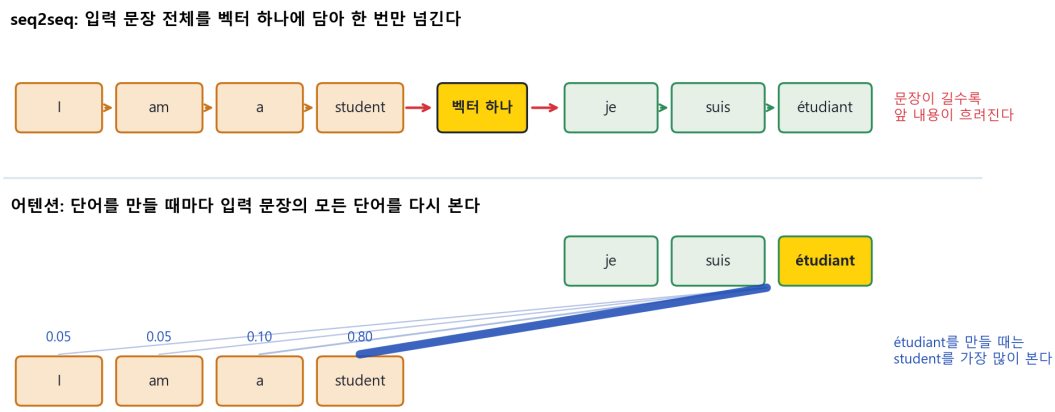


그림 1 위: 16장의 seq2seq. 아래: 어텐션. étudiant를 만들 때 입력의 네 단어를 다시 보고 student에 가장 큰 비중을 둔다(설명용 값)

2) 닷-프로덕트 어텐션의 여섯 단계

정의

디코더가 시점 t의 단어를 예측하려는 순간입니다. 디코더 LSTM이 이미 은닉 상태 s_t를 만들었습니다. 인코더와 디코더의 은닉 크기는 같다고 둡니다.

① **어텐션 스코어.** s_t가 인코더의 각 은닉 상태 h_i와 얼마나 비슷한지를 내적으로 잽니다. 값 하나(스칼라)가 나옵니다.

$$\text{score}(s_t,\ h_i) = s_t^T h_i, \qquad e^t = [\, s_t^T h_1,\ \ldots,\ s_t^T h_N \,]$$

② **어텐션 분포.** 스코어에 소프트맥스를 씌워 합이 1인 비율로 바꿉니다. 각 값을 어텐션 가중치라고 합니다.

$$\alpha^t = \text{softmax}(e^t)$$

③ **어텐션 값.** 그 비율로 인코더의 은닉 상태들을 가중합합니다. 이 벡터는 "지금 시점에 필요한 입력 문장의 요약"이라서 이것도 컨텍스트 벡터라고 부릅니다. 16장의 컨텍스트 벡터는 문장마다 하나였지만, 이것은 **시점마다 새로** 만들어집니다.

$$a_t = \sum_{i=1}^{N} \alpha_i^t h_i$$

④ **이어 붙이기.** 어텐션 값 a_t와 디코더의 은닉 상태 s_t를 이어 붙입니다(concatenate): $v_t = [a_t; s_t]$. 크기가 두 배가 됩니다.

⑤ **한 번 더 가공.** v_t에 가중치 행렬을 곱하고 tanh를 씌워 원래 크기로 줄입니다.

$$\tilde{s}_t = \tanh(W_c\,[a_t; s_t] + b_c)$$

⑥ **출력층.** $\tilde{s}_t$를 출력층에 넣어 단어의 확률을 얻습니다. 16장에서는 s_t를 바로 넣었습니다.

$$\hat{y}_t = \text{softmax}(W_y\,\tilde{s}_t + b_y)$$

e^t 스코어의 모음, α^t 어텐션 분포, a_t 어텐션 값, W_c, b_c, W_y, b_y 학습되는 가중치와 편향입니다. ①~③에는 학습할 가중치가 없습니다. 내적, 소프트맥스, 가중합뿐입니다.

왜 내적이 "비슷한 정도"가 될까? 내적은 두 벡터가 같은 방향을 가리키고 클수록 커집니다(11-04의 코사인 유사도의 분자). 모델은 학습을 통해 "étudiant를 만들 차례의 s_t"와 "student를 읽은 h_4"가 비슷한 방향을 갖도록 인코더와 디코더의 가중치를 조정합니다. 어디를 볼지는 사람이 정해 주지 않고 학습으로 정해집니다.

스코어 함수의 종류. ①의 스코어를 구하는 방법에 따라 어텐션의 이름이 달라집니다. ②~③은 모두 같습니다.

이름	스코어 함수	학습할 가중치	제안
dot	$s_t^T h_i$	없음	루옹 (2015)
scaled dot	$s_t^T h_i \div \sqrt{n}$	없음	트랜스포머 (2017, 18장)
general	$s_t^T W_a h_i$	W_a	루옹 (2015)
concat	$W_a^T \tanh(W_b s_t + W_c h_i)$	W_a, W_b, W_c	바다나우 (2015, 17-02)

예시 적용

은닉 크기 3인 설명용 값으로 계산합니다(실습 1). 디코더가 je, suis를 만든 뒤 세 번째 단어를 만들려는 시점입니다.

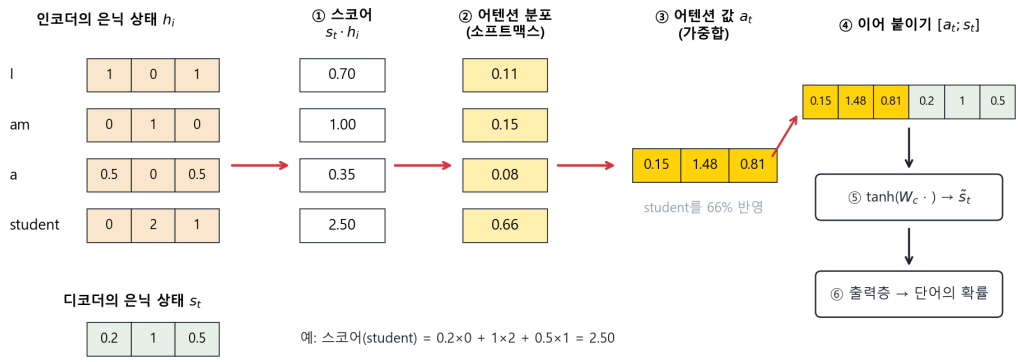


그림 2 닷-프로덕트 어텐션을 숫자로 따라가기(설명용 값, 실습 1의 실제 계산)

- ① 스코어: I 0.70, am 1.00, a 0.35, student 2.50. student는 $0.2 \times 0 + 1 \times 2 + 0.5 \times 1 = 2.50$입니다.
- ② 어텐션 분포: 0.11, 0.15, 0.08, 0.66 (합 1). student를 66% 봅니다.
- ③ 어텐션 값: [0.15, 1.48, 0.81]. student의 은닉 상태 [0, 2, 1]에 가깝게 끌려갔습니다.
- ④ 이어 붙이면 6칸, ⑤ tanh를 지나면 다시 3칸, ⑥ 출력층에서 단어마다의 확률이 됩니다.
- 디코더의 여러 시점을 행렬로 한 번에 계산해도 첫 행이 같은 값입니다(True). 둘째 시점의 분포는 [0.43, 0.16, 0.26, 0.16]으로, 시점마다 보는 곳이 달라집니다.

비유

원문을 펴 놓고 하는 번역

16장의 통역사는 원문을 듣고 메모 한 장만 들고 말했습니다. 이번에는 원문을 책상에 펴 놓습니다. 단어를 하나 옮길 때마다 원문 전체를 훑어보되, 지금 옮길 부분에 눈이 오래 머뭅니다. "지금 내가 하려는 말"을 떠올리고(질문), 원문의 각 단어가 그것과 얼마나 관련 있는지 가늠해(점수), 관련 있는 단어들의 뜻을 머릿속에서 섞어(가중합) 다음 단어를 정합니다.

지금 하려는 말	= = Query (디코더의 은닉 상태)
원문의 단어들	= = Key, Value (인코더의 은닉 상태들)
눈이 머무는 시간의 비율	= = 어텐션 분포
섞어서 떠올린 뜻	= = 어텐션 값

실습 1

닷-프로덕트 어텐션의 여섯 단계를 숫자로 계산하기

```
import torch

# 인코더의 은닉 상태 4개 (I, am, a, student)와 디코더의 현재 은닉 상태. 설명용 값
words = ["I", "am", "a", "student"]
H = torch.tensor([[1.0, 0.0, 1.0],       # h1: I
                  [0.0, 1.0, 0.0],       # h2: am
                  [0.5, 0.0, 0.5],       # h3: a
                  [0.0, 2.0, 1.0]])      # h4: student
s_t = torch.tensor([0.2, 1.0, 0.5])      # 디코더가 세 번째 단어를 만들려는 시점

# 1) 어텐션 스코어: s_t와 각 h_i의 내적
scores = torch.stack([s_t @ h for h in H])
print("① 스코어      :", [f"{v:.2f}" for v in scores.tolist()])

# 2) 어텐션 분포: 소프트맥스로 합이 1인 비율로
weights = torch.softmax(scores, dim=0)
print("② 어텐션 분포 :", [f"{v:.2f}" for v in weights.tolist()],
      " 합:", round(weights.sum().item(), 2))

# 3) 어텐션 값: 그 비율로 h_i를 섞는다 (가중합)
a_t = sum(w * h for w, h in zip(weights, H))
print("③ 어텐션 값   :", [f"{v:.2f}" for v in a_t.tolist()])

# 4) 어텐션 값과 디코더의 은닉 상태를 이어 붙인다
v_t = torch.cat([a_t, s_t])
print("④ 이어 붙이기 :", tuple(v_t.shape), [f"{v:.2f}" for v in v_t.tolist()])

# 5) 가중치 행렬을 곱하고 tanh → 6) 출력층에서 단어마다의 점수로
torch.manual_seed(0)
W_c = torch.nn.Linear(6, 3)              # (2H → H)
W_y = torch.nn.Linear(3, 5)              # (H → 단어 집합 크기 5)
s_tilde = torch.tanh(W_c(v_t))
y_hat = torch.softmax(W_y(s_tilde), dim=0)
print("⑤ tanh 뒤     :", tuple(s_tilde.shape),
      " ⑥ 단어별 확률:", tuple(y_hat.shape))

# 행렬로 한꺼번에: 디코더의 모든 시점(T개)에 대해 한 번에 계산한다
S = torch.stack([s_t, torch.tensor([1.0, 0.0, 0.0])])    # 디코더 시점 2개 (2, 3)
score_matrix = S @ H.T                   # (디코더 시점 수, 인코더 시점 수)
attn = torch.softmax(score_matrix, dim=-1)
A = attn @ H                             # (디코더 시점 수, 은닉 크기)
```

```
print("\n행렬 계산: 스코어", tuple(score_matrix.shape),
      "→ 어텐션 값", tuple(A.shape))
print("첫 행이 위에서 구한 값과 같은가?", torch.allclose(A[0], a_t))
print("둘째 시점의 분포:", [f"{v:.2f}" for v in attn[1].tolist()])
```

실행 결과

```
① 스코어       : ['0.70', '1.00', '0.35', '2.50']
② 어텐션 분포 : ['0.11', '0.15', '0.08', '0.66']  합: 1.0
③ 어텐션 값    : ['0.15', '1.48', '0.81']
④ 이어 붙이기 : (6,) ['0.15', '1.48', '0.81', '0.20', '1.00', '0.50']
⑤ tanh 뒤      : (3,)  ⑥ 단어별 확률: (5,)

행렬 계산: 스코어 (2, 4) → 어텐션 값 (2, 3)
첫 행이 위에서 구한 값과 같은가? True
둘째 시점의 분포: ['0.43', '0.16', '0.26', '0.16']
```

직접 해 보기 `s_t`를 `[1.0, 0.0, 1.0]`(I의 은닉 상태와 같은 값)으로 바꾸면 어텐션 분포는 어디에 몰리나요? 먼저 스코어를 손으로 계산해 보세요.

확인 문제

17-01 확인 문제 한 명씩 돌아가며 풀고, 푼 사람 이름을 적어 주세요

1 어텐션은 인코더의 마지막 은닉 상태 하나만 쓴다. (O / X) 이름 _____

2 어텐션 분포의 값을 모두 더하면 1이다. (O / X) 이름 _____

3 seq2seq + 어텐션에서 Query는? 이름 _____

① 인코더의 은닉 상태들 ② 디코더의 현재 은닉 상태 ③ 입력 단어의 임베딩

4 닷-프로덕트 어텐션에서 스코어를 구하는 계산은? 이름 _____

① 두 벡터의 내적 ② 두 벡터의 차 ③ 두 벡터를 이어 붙이기

5 스코어에 _____ 함수를 씌우면 어텐션 분포가 된다. 이름 _____

6 어텐션 값은 어텐션 분포를 비율로 한 인코더 은닉 상태들의 _____합이다. 이름 _____

7 어텐션의 스코어, 분포, 값을 원문을 펴 놓고 하는 번역에 빗대어 설명해 보세요. 이름 _____

원문: 17-01 어텐션 메커니즘 — https://wikidocs.net/65775 (숫자 예는 손으로 따라갈 수 있게 은닉 크기 3의 설명용 값으로 새로 만들었습니다)

17-02 바다나우 어텐션

핵심 한 줄

바다나우 어텐션은 디코더의 이전 시점의 은닉 상태를 Query로 쓰고, 작은 신경망으로 스코어를 구하며, 어텐션 값을 입력 임베딩과 이어 붙여 디코더 LSTM의 입력으로 넣는다.

왜 배우나

어텐션을 처음 제안한 것은 바다나우의 논문(2015)입니다. 17-01의 닷-프로덕트(루옹) 어텐션은 그것을 단순하게 만든 것입니다. 두 방법을 나란히 놓으면 "어텐션의 뼈대(스코어 → 분포 → 가중합)는 같고, 스코어를 구하는 방법과 결과를 끼워 넣는 자리만 다르다"는 것이 보입니다.

1) 바다나우 어텐션의 계산

정의

Query가 다릅니다. 시점 t의 단어를 예측할 때, 아직 s_t를 만들기 전이므로 **이전 시점의 은닉 상태** s_{t-1}을 Query로 씁니다. Key와 Value는 같습니다(인코더의 모든 은닉 상태).

① **스코어.** 내적 대신 작은 신경망을 씁니다.

$$\text{score}(s_{t-1},\ h_i) = W_a^T \tanh(W_b\, s_{t-1} + W_c\, h_i)$$

- W_b: 디코더의 은닉 상태를 변환하는 가중치 행렬
- W_c: 인코더의 은닉 상태를 변환하는 가중치 행렬
- W_a: 둘을 더해 tanh를 씌운 벡터를 점수 하나로 바꾸는 가중치
- 셋 다 학습됩니다. "어떤 것끼리 관련 있는가"를 내적이 아니라 학습된 함수로 판단합니다.

② **어텐션 분포,** ③ **어텐션 값(컨텍스트 벡터).** 17-01과 같습니다. 소프트맥스를 씌우고 가중합합니다.

④ **디코더 LSTM의 입력으로.** 컨텍스트 벡터를 현재 입력 단어의 임베딩 벡터와 이어 붙여 LSTM 셀에 넣습니다. LSTM은 이 입력과 s_{t-1}로 s_t를 만들고, s_t가 출력층으로 갑니다.

	닷-프로덕트(루옹) 어텐션	바다나우 어텐션
Query	현재 시점의 은닉 상태 s_t	이전 시점의 은닉 상태 s_{t-1}
스코어	내적 $s_t^T h_i$	$W_a^T \tanh(W_b s_{t-1} + W_c h_i)$
스코어의 가중치	없음	W_a, W_b, W_c
어텐션 값을 쓰는 자리	LSTM **뒤**: s_t와 이어 붙여 출력층으로	LSTM **앞**: 입력 임베딩과 이어 붙여 LSTM으로
같은 것	스코어 → 소프트맥스(분포) → 가중합(어텐션 값)	

어느 쪽이 좋을까? 바다나우 방식은 스코어를 학습된 함수로 구해 표현력이 크지만 계산이 많습니다. 닷-프로덕트는 가중치가 없어 빠르고, 은닉 크기가 같아야 한다는 조건이 붙습니다. 트랜스포머는 닷-프로덕트 쪽을 택했습니다.

예시 적용

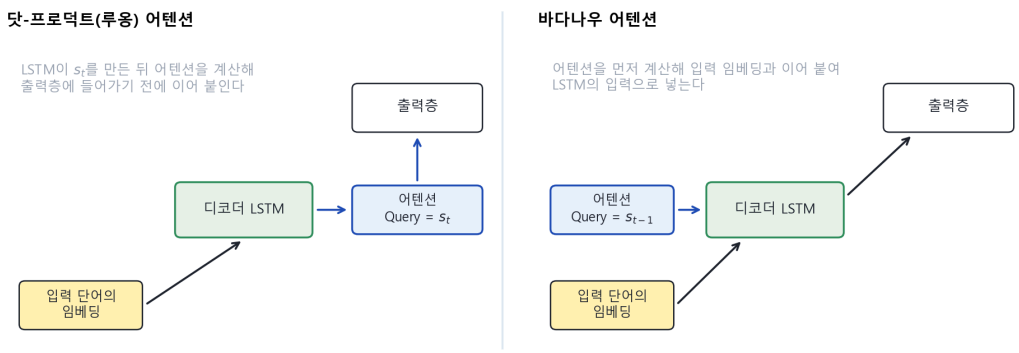


그림 3 어텐션을 끼워 넣는 자리의 차이. 루옹은 LSTM 뒤에서, 바다나우는 LSTM 앞에서 쓴다

은닉 크기 4, 임베딩 8, 스코어 신경망의 중간 크기 6으로 모양을 확인합니다(실습 2, 학습되지 않은 값).

단계	모양
$W_b s_{t-1}$	(6,)
$W_c H$ (인코더 은닉 상태 4개를 한꺼번에)	(4, 6)
더하고 tanh	(4, 6)
W_a를 곱한 스코어	(4,) 입력 단어마다 점수 하나
컨텍스트 벡터	(4,)
LSTM의 입력 = 컨텍스트 4 + 임베딩 8	(12,)

같은 s와 H에 세 가지 스코어 함수를 적용하면 어텐션 분포가 각각 [0.69, 0.06, 0.20, 0.05](dot), [0.43, 0.19, 0.23, 0.15](general), [0.26, 0.24, 0.24, 0.26](concat)으로 다릅니다. 학습할 가중치는 0개, 16개, 54개입니다. 학습 전이라 숫자에 뜻은 없고, "스코어 함수에 따라 보는 곳이 달라진다"는 점만 봅니다.

비유

말하기 전에 원문을 보느냐, 말을 떠올린 뒤에 보느냐

한 통역사는 다음 말을 떠올리기 **전에** 원문을 먼저 훑고, 본 내용을 바탕으로 말을 만듭니다. 다른 통역사는 일단 다음 말을 대강 떠올린 **뒤에** 원문을 훑어 다듬습니다. 둘 다 원문을 다시 본다는 점은 같습니다.

말을 떠올리기 전에 원문 보기 = = 바다나우 (LSTM의 입력으로)

말을 떠올린 뒤에 원문 보기 = = 루옹 (LSTM의 출력에 덧붙여)

실습 2

바다나우 어텐션의 모양과 세 가지 스코어 함수

```
import torch
import torch.nn as nn

torch.manual_seed(0)
hidden = 4
H = torch.randn(4, hidden)                # 인코더의 은닉 상태 4개 (4, 4)
s_prev = torch.randn(hidden)              # 디코더의 이전 시점 은닉 상태 s_{t-1}
x_t = torch.randn(8)                      # 현재 입력 단어의 임베딩 (8칸)

# 1) 바다나우(concat) 스코어: W_a^T tanh(W_b s_{t-1} + W_c h_i)
W_b = nn.Linear(hidden, 6, bias=False)    # s_{t-1} → 6칸
W_c = nn.Linear(hidden, 6, bias=False)    # h_i → 6칸
W_a = nn.Linear(6, 1, bias=False)         # 6칸 → 점수 하나
inner = torch.tanh(W_b(s_prev) + W_c(H)) # (4, 6): s는 한 개, H는 네 개
scores = W_a(inner).squeeze(-1)           # (4,)
print("W_b s:", tuple(W_b(s_prev).shape), " W_c H:", tuple(W_c(H).shape),
      " tanh 뒤:", tuple(inner.shape), " 스코어:", tuple(scores.shape))

# 2) 어텐션 분포 → 3) 어텐션 값(컨텍스트 벡터). 이 두 단계는 닷-프로덕트와 같다
weights = torch.softmax(scores, dim=0)
context = weights @ H                     # (4,)
print("어텐션 분포:", [f"{v:.2f}" for v in weights.tolist()],
      " 컨텍스트 벡터:", tuple(context.shape))

# 4) 컨텍스트 벡터를 입력 임베딩과 이어 붙여 LSTM 셀의 입력으로 쓴다
cell = nn.LSTMCell(8 + hidden, hidden)    # 입력 크기 = 임베딩 8 + 컨텍스트 4
lstm_input = torch.cat([context, x_t])
state = (s_prev.unsqueeze(0), torch.zeros(1, hidden))
s_t, c_t = cell(lstm_input.unsqueeze(0), state)
print("LSTM 입력:", tuple(lstm_input.shape),
      "→ 새 은닉 상태 s_t:", tuple(s_t.shape))

# 5) 스코어 함수 세 가지를 같은 s, H에 적용해 본다
general = nn.Linear(hidden, hidden, bias=False)
table = {"dot      s·h": H @ s_prev,
         "general  s·(W h)": general(H) @ s_prev,
         "concat   W_a·tanh(W_b s + W_c h)": scores}
print("\n스코어 함수                          학습할 가중치   어텐션 분포")
for (name, score), n in zip(table.items(), [0, 16, 54]):
    dist = [f"{v:.2f}" for v in torch.softmax(score, dim=0).tolist()]
    print(f"{name:34s} {n:5d}개       {dist}")
```

실행 결과

```
W_b s: (6,)  W_c H: (4, 6)  tanh 뒤: (4, 6)  스코어: (4,)
어텐션 분포: ['0.26', '0.24', '0.24', '0.26']  컨텍스트 벡터: (4,)
LSTM 입력: (12,) → 새 은닉 상태 s_t: (1, 4)

스코어 함수                          학습할 가중치   어텐션 분포
dot      s·h                              0개       ['0.69', '0.06', '0.20',
```

```
'0.05']
general  s·(W h)                      16개       ['0.43', '0.19', '0.23',
'0.15']
concat   W_a·tanh(W_b s + W_c h)      54개       ['0.26', '0.24', '0.24',
'0.26']
```

직접 해 보기 `W_b`, `W_c`의 출력 크기 6을 10으로 바꾸면 concat 스코어의 가중치는 몇 개가 되나요?

확인 문제

17-02 확인 문제 한 명씩 돌아가며 풀고, 푼 사람 이름을 적어 주세요

1 바다나우 어텐션의 스코어 함수에는 학습되는 가중치가 있다. (O / X) 이름 _____

2 바다나우 어텐션과 닷-프로덕트 어텐션은 어텐션 분포를 구하는 방법(소프트맥스)이 다르다. (O / X) 이름 _____

3 바다나우 어텐션의 Query는? 이름 _____

① s_t ② s_{t-1} ③ h_N

4 바다나우 어텐션에서 컨텍스트 벡터를 이어 붙이는 대상은? 이름 _____

① 현재 입력 단어의 임베딩 ② 출력층의 점수 ③ 인코더의 입력

5 닷-프로덕트 어텐션은 제안한 사람의 이름을 따서 _____ 어텐션이라고도 한다. 이름 _____

6 바다나우 스코어 함수에서 두 항을 더한 뒤 씌우는 활성화 함수는 _____이다. 이름 _____

7 두 어텐션의 같은 점과 다른 점을 하나씩 말해 보세요. 이름 _____

원문: 17-02 바다나우 어텐션 — https://wikidocs.net/216493 (비교 표와 모양을 확인하는 실습은 이 자료에서 추가했습니다)

17-03 어텐션 메커니즘을 이용한 번역기 구현

핵심 한 줄

16장의 번역기에서 인코더의 모든 시점의 출력을 버리지 않고 디코더에 넘겨, 디코더가 시점마다 닷-프로덕트 어텐션으로 입력 문장을 다시 보게 하면 같은 데이터로도 번역이 나아진다.

왜 배우나

식으로 본 어텐션을 실제 번역기에 넣어 봅니다. 코드에서 바뀌는 것은 몇 줄뿐인데, 그 몇 줄이 무엇인지 알면 "어텐션을 더했다"는 말의 뜻이 분명해집니다. 같은 데이터, 같은 크기의 모델로 16장과 비교하고, 모델이 실제로 어디를 보고 번역했는지도 꺼내 봅니다.

1) 16장의 번역기에서 달라지는 것

정의

	seq2seq (16장)	seq2seq + 어텐션
인코더가 넘기는 것	마지막 (hidden, cell)	마지막 (hidden, cell) + **모든 시점의 출력** (B, L, H)
디코더의 계산	LSTM → 선형층	LSTM → **어텐션(①~⑤)** → 선형층
추가된 가중치		W_c: (2H → H) 하나
패딩	손실에서만 뺌	어텐션에서도 가림 (패딩 마스크)

$$\underbrace{S}_{(B,T,H)} \times \underbrace{H_{enc}^{T}}_{(B,H,L)} = \underbrace{\text{scores}}_{(B,T,L)} \xrightarrow{\text{softmax}} \underbrace{\alpha}_{(B,T,L)} \qquad \alpha \times \underbrace{H_{enc}}_{(B,L,H)} = \underbrace{A}_{(B,T,H)}$$

B 배치 크기, T 출력 문장의 길이, L 입력 문장의 길이, H 은닉 크기입니다. S는 디코더의 모든 시점의 은닉 상태, H_{enc}는 인코더의 모든 시점의 은닉 상태입니다. 교사 강요 덕분에 디코더의 모든 시점을 한꺼번에 계산할 수 있어, 어텐션도 행렬 곱 두 번(`torch.bmm`)으로 끝납니다. 스코어의 모양 (B, T, L)은 "출력 단어마다, 입력 단어마다 점수 하나"입니다.

패딩 마스크. 입력 문장의 패딩 자리는 뜻이 없으므로, 스코어에 아주 작은 값(-1e9)을 넣어 소프트맥스 뒤에 비율이 0이 되게 합니다. 18장의 패딩 마스크와 같은 것입니다.

교재의 구현은 디코더의 처음 상태로 어텐션을 한 번만 계산해 모든 시점에 같은 컨텍스트 벡터를 쓰는 간단한 방식입니다. 이 자료는 17-01의 식대로 시점마다 어텐션을 계산하도록 구현했습니다.

예시 적용

16장과 같은 데이터(문장 쌍 33,000개), 같은 크기(임베딩 256, 은닉 256), 같은 10 에포크로 학습했습니다(실습 4).

테스트 문장 500개	seq2seq (16장)	seq2seq + 어텐션
가장 좋은 검증 손실	1.6576	1.4507
검증 정확도 (교사 강요)	0.7007	0.7295
BLEU	0.2471	0.3007
정답과 완전히 같은 번역	34개	51개

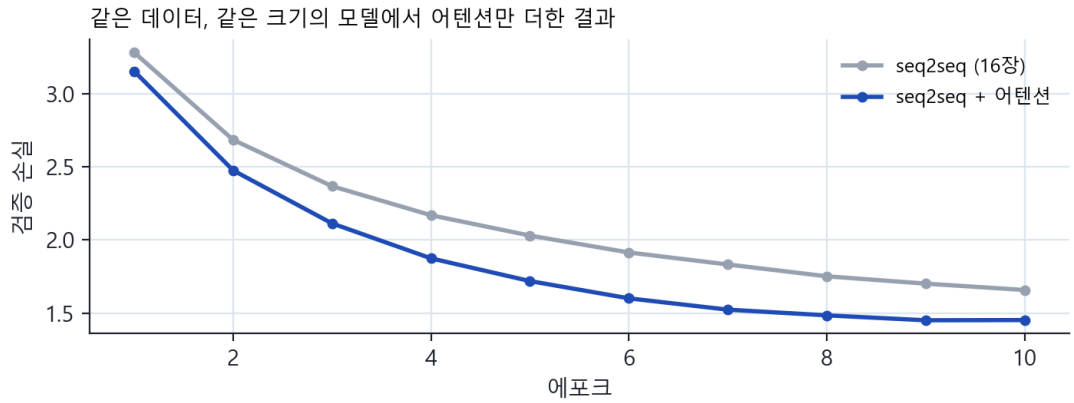


그림 4 에포크마다의 검증 손실. 3 에포크부터 어텐션을 쓴 모델이 계속 낮다

- **모든 숫자가 나아졌습니다.** BLEU가 0.2471에서 0.3007로, 완전히 같은 번역이 34개에서 51개로 늘었습니다. 교재도 검증 정확도가 0.7180에서 0.7334로 올랐습니다.
- **10 에포크에서 검증 손실이 다시 올랐습니다**(1.4507 → 1.4522). 저장된 것은 9 에포크의 모델입니다.

입력	정답	16장의 번역	어텐션을 쓴 번역
get out of the truck .	sortez du camion !	sortez de la porte ! (문에서 나가라)	sors de la camionnette ! (트럭에서 나가라)
the door creaked .	la porte grinca .	la porte s ouvrit de la porte . (반복)	la porte s est ouverte . (문이 열렸다)
we all hate tom .	nous detestons tous tom .	nous allons tous tom . (간다)	nous aimons tous tom . (좋아한다)

- truck을 "문"으로 바꾸던 것이 camionnette(소형 트럭)로 맞게 나왔습니다. 정답의 camion과 단어가 달라 BLEU에서는 점수를 받지 못하지만 뜻은 맞습니다.
- 같은 말을 반복하던 문장이 사라졌습니다. 다만 creaked(삐걱거렸다)는 여전히 "열렸다"로 옮겼습니다.
- **여전히 틀립니다.** hate를 aimons(좋아한다)로 옮겨 뜻이 정반대가 됐습니다. hate와 love는 쓰이는 자리가 비슷해 벡터가 가깝고(12장), 데이터가 적어 구분을 배우지 못했습니다.

모델은 어디를 봤을까: 예상과 다른 결과. "get out of the truck ."을 번역할 때의 어텐션 분포를 꺼냈습니다.

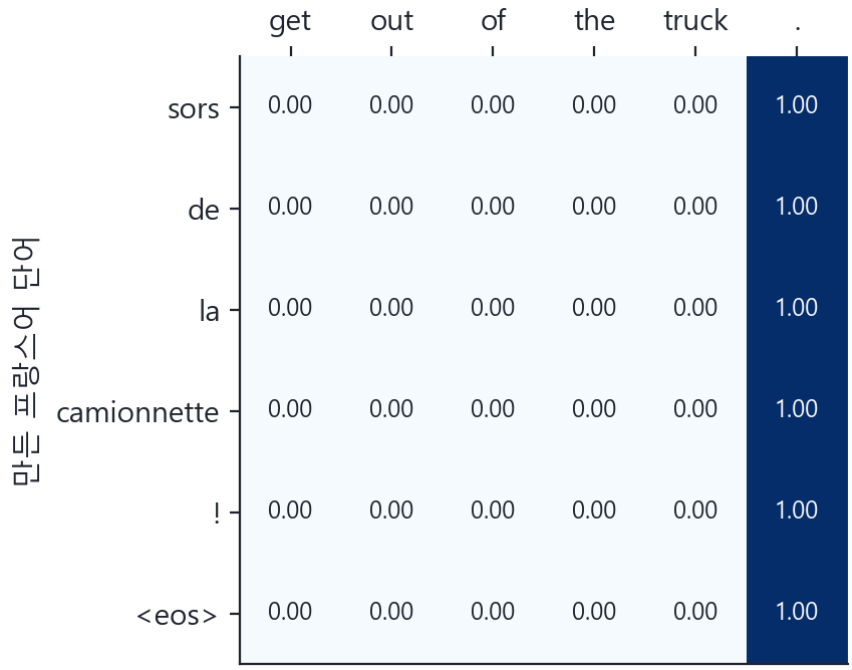


그림 5 실습 4의 실제 어텐션 지도. 모든 단어를 만들 때 마지막 토큰(.)만 본다

- camionnette를 만들 때 truck을 볼 것이라 기대했지만, **모든 단어에서 마지막 토큰 "."에 가중치 1.00**이 갔습니다. 17-01의 그림 같은 지도가 나오지 않았습니다.
- 테스트 문장 500개 전체로 확인해도 같습니다. 가장 많이 본 자리가 마지막 토큰인 경우가 88.3%이고, 마지막 토큰에 준 가중치는 평균 0.87입니다.

왜 이렇게 됐을까? 측정한 것과 그 해석입니다.

- **스코어가 너무 큽니다.** 256칸짜리 벡터의 내적이라 스코어가 평균 15, 마지막 토큰은 21쯤 나옵니다. 차이가 6이면 소프트맥스 뒤에는 $e^6 \approx 400$배 차이가 되어 한 곳에 거의 1이 몰립니다. 18장의 트랜스포머가 스코어를 $\sqrt{d_k}$로 나누는 이유가 이것입니다.
- **마지막 은닉 상태가 가장 큽니다.** 인코더 은닉 상태의 크기(노름)를 재면 마지막 토큰이 10.07, 나머지가 7.23입니다. LSTM이 읽어 갈수록 정보가 쌓이기 때문이고, 내적은 벡터가 클수록 커집니다.
- **문장이 짧습니다.** 입력이 최대 8토큰이라 마지막 은닉 상태 하나에 문장 전체가 이미 담깁니다. 그곳만 봐도 번역에 필요한 정보를 얻을 수 있습니다.

그렇다면 왜 성능은 올랐을까? 이 실습으로 확인한 것은 "성능이 올랐다"와 "어텐션이 마지막 토큰에 쏠렸다"까지입니다. 단어를 만들 때마다 입력 문장의 요약(마지막 은닉 상태)을 다시 받아 쓰게 된 것이 도움이 됐을 것으로 짐작하지만, 따로 실험해 확인하지는 않았습니다. 16장의 디코더는 그 요약을 처음에 한 번만 받았습니다. 어텐션이 단어를 골라 보는 효과는 문장이 길고 데이터가 많을 때 나타납니다(18장의 실습에서는 ne가 not을 보는 지도가 나왔습니다).

비유

메모 한 장에서, 형광펜을 든 원문으로

16장의 통역사는 메모 한 장만 받았습니다. 이번 통역사는 원문 전체를 받고, 단어를 말할 때마다 원문의 관련된 곳에 형광펜을 칠합니다. 통역이 끝난 뒤 형광펜 자국을 보면 "이 단어를 말할 때 원문의 어디를 봤는지"를 알 수 있습니다. 어텐션 지도가 그 자국입니다.

메모 한 장	= = 인코더의 마지막 상태 (16장)
원문 전체	= = 인코더의 모든 시점의 출력
형광펜의 진하기	= = 어텐션 가중치
형광펜 자국	= = 어텐션 지도

실습 3

데이터 준비 (16장의 실습 2와 같음)

```
# 16장과 같은 데이터 준비 (영어-프랑스어 문장 쌍 33,000개 → 정수 텐서)
import io
import re
import unicodedata
import urllib.request
import zipfile
from collections import Counter

import torch

# 교재의 주소(manythings.org)는 프로그램으로 받으면 막힐 때가 있어, 같은 자료의
# 다른 보관 주소를 쓴다
url = "https://storage.googleapis.com/download.tensorflow.org/data/fra-eng.zip"
zipfile.ZipFile(io.BytesIO(urllib.request.urlopen(url).read())).extract(
    "fra.txt")
num_samples = 33000                                  # 교재와 같이 앞 33,000쌍만

def preprocess(sentence):
    # 악센트 제거(é → e), 소문자로, 구두점 앞에 공백, 그 밖의 기호 제거
    s = unicodedata.normalize("NFD", sentence.lower())
    s = "".join(c for c in s if unicodedata.category(c) != "Mn")
    s = re.sub(r"([?.!,])", r" \1", s)
    s = re.sub(r"[^a-z!.?]+", " ", s)
    return s.split()

pairs = []
for line in open("fra.txt", encoding="utf8").read().split("\n")[:num_samples]:
    en, fr = line.split("\t")[:2]
    pairs.append((preprocess(en), preprocess(fr)))
print("문장 쌍:", len(pairs))
for i in [0, 100, 30000]:
    print(f"  {' '.join(pairs[i][0]):28s} ↔ {' '.join(pairs[i][1])}")

# 디코더의 입력은 앞에 <sos>를, 정답은 뒤에 <eos>를 붙인다
enc_in = [en for en, _ in pairs]
dec_in = [["<sos>"] + fr for _, fr in pairs]
dec_out = [fr + ["<eos>"] for _, fr in pairs]

def build_vocab(sentences):
    counts = Counter(w for s in sentences for w in s)
    words = sorted(counts, key=counts.get, reverse=True)
    return {"<PAD>": 0, "<UNK>": 1, **{w: i + 2 for i, w in enumerate(words)}}

src_vocab = build_vocab(enc_in)
tar_vocab = build_vocab(dec_in + dec_out)            # <sos>, <eos>도 들어간다
print("영어 단어 집합:", len(src_vocab), " 프랑스어 단어 집합:", len(tar_vocab))

def encode(sentences, vocab):
```

```
    length = max(len(s) for s in sentences)
    out = torch.zeros(len(sentences), length, dtype=torch.long)
    for i, s in enumerate(sentences):
        out[i, :len(s)] = torch.tensor([vocab[w] for w in s])
    return out

X, Y_in, Y_out = encode(enc_in, src_vocab), encode(dec_in, tar_vocab), encode(
    dec_out, tar_vocab)
print("인코더 입력:", tuple(X.shape), " 디코더 입력:", tuple(Y_in.shape),
      " 정답:", tuple(Y_out.shape))
tar_words = {i: w for w, i in tar_vocab.items()}
print("디코더 입력 예:", [tar_words[i] for i in Y_in[30000].tolist() if i])
print("그 정답       :", [tar_words[i] for i in Y_out[30000].tolist() if i])

# 섞은 뒤 10%를 테스트용으로 뗀다
torch.manual_seed(0)
order = torch.randperm(num_samples)
n_test = num_samples // 10
test, train = order[:n_test], order[n_test:]
data = {"src_vocab": src_vocab, "tar_vocab": tar_vocab}
for name, idx in [("train", train), ("test", test)]:
    data[name] = (X[idx], Y_in[idx], Y_out[idx])
torch.save(data, "seq2seq_data.pt")
print("학습:", len(train), " 테스트:", len(test))
```

실행 결과

```
문장 쌍: 33000
  go .                          ↔ va !
  come in .                     ↔ entrez !
  sing a song with me .         ↔ chante une chanson avec moi .
영어 단어 집합: 4674  프랑스어 단어 집합: 8087
인코더 입력: (33000, 8)  디코더 입력: (33000, 16)  정답: (33000, 16)
디코더 입력 예: ['<sos>', 'chante', 'une', 'chanson', 'avec', 'moi', '.']
그 정답       : ['chante', 'une', 'chanson', 'avec', 'moi', '.', '<eos>']
학습: 29700  테스트: 3300
```

직접 해 보기 16장에서 이미 `seq2seq_data.pt`를 만들어 두었다면 이 실습은 건너뛰어도 됩니다.

실습 4 어텐션을 더한 번역기 학습하고, 어디를 봤는지 꺼내 보기

```
# 앞 실습(데이터 준비)을 먼저 실행해 둔다 (CPU로 약 20분)
import torch
import torch.nn as nn
from nltk.translate.bleu_score import corpus_bleu
from torch.utils.data import DataLoader, TensorDataset

torch.manual_seed(0)
data = torch.load("seq2seq_data.pt")
src_vocab, tar_vocab = data["src_vocab"], data["tar_vocab"]
src_words = {i: w for w, i in src_vocab.items()}
tar_words = {i: w for w, i in tar_vocab.items()}
SOS, EOS = tar_vocab["<sos>"], tar_vocab["<eos>"]

class AttentionSeq2Seq(nn.Module):
    def __init__(self, embedding_dim=256, hidden_units=256):
        super().__init__()
        embed = lambda vocab: nn.Embedding(len(vocab), embedding_dim, padding_i
dx=0)
        self.src_embedding, self.tar_embedding = embed(src_vocab), embed(tar_vo
cab)
        self.encoder = nn.LSTM(embedding_dim, hidden_units, batch_first=True)
        self.decoder = nn.LSTM(embedding_dim, hidden_units, batch_first=True)
        self.W_c = nn.Linear(hidden_units * 2, hidden_units)
        self.fc = nn.Linear(hidden_units, len(tar_vocab))
```

```
    def encode(self, src):
        # 16장과 달리 인코더의 모든 시점의 은닉 상태(enc_out)도 쓴다
        enc_out, state = self.encoder(self.src_embedding(src))
        return enc_out, state, src == 0              # 패딩 자리 표시

    def decode(self, dec_input, enc_out, state, pad_mask):
        # 디코더의 모든 시점의 은닉 상태 s: (B, T, H)
        s, state = self.decoder(self.tar_embedding(dec_input), state)
        # ① 스코어: 디코더의 모든 시점 × 인코더의 모든 시점의 내적 (B, T, L)
        scores = torch.bmm(s, enc_out.transpose(1, 2))
        # 패딩은 보지 않게 가린다
        scores = scores.masked_fill(pad_mask.unsqueeze(1), -1e9)
        weights = torch.softmax(scores, dim=-1)      # ② 어텐션 분포
        a = torch.bmm(weights, enc_out)              # ③ 어텐션 값 (B, T, H)
        s_tilde = torch.tanh(self.W_c(torch.cat([a, s], dim=-1)))   # ④ ⑤
        return self.fc(s_tilde), state, weights      # ⑥ 단어마다의 점수

    def forward(self, src, dec_input):
        enc_out, state, pad_mask = self.encode(src)
        return self.decode(dec_input, enc_out, state, pad_mask)[0]

model = AttentionSeq2Seq()
criterion = nn.CrossEntropyLoss(ignore_index=0)
optimizer = torch.optim.Adam(model.parameters())
X, Y_in, Y_out = data["train"]
n_valid = 3000
train_set = TensorDataset(X[:-n_valid], Y_in[:-n_valid], Y_out[:-n_valid])
valid = (X[-n_valid:], Y_in[-n_valid:], Y_out[-n_valid:])

def evaluate(src, dec_input, target):
    model.eval()
    with torch.no_grad():
        logits = model(src, dec_input)
    loss = criterion(logits.view(-1, logits.size(-1)), target.view(-1)).item()
    mask = target != 0
    return loss, (logits.argmax(-1) == target)[mask].float().mean().item()

history, best = [], float("inf")
for epoch in range(1, 11):
    model.train()
    total = 0.0
    loader = DataLoader(train_set, batch_size=128, shuffle=True)
    for src, dec_input, target in loader:
        logits = model(src, dec_input)
        loss = criterion(logits.view(-1, logits.size(-1)), target.view(-1))
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        total += loss.item() * len(src)
    val_loss, val_acc = evaluate(*valid)
    history.append((total / len(train_set), val_loss, val_acc))
    saved = ""
    if val_loss < best:
        best, saved = val_loss, "  ← 저장"
        torch.save(model.state_dict(), "attention.pth")
    print(f"에포크 {epoch:2d}: 학습 손실 {history[-1][0]:.4f} |",
          f"검증 손실 {val_loss:.4f}, 정확도 {val_acc:.4f}{saved}")
model.load_state_dict(torch.load("attention.pth"))

def translate(src_ids, max_len=20):
    model.eval()
    with torch.no_grad():
        enc_out, state, pad_mask = model.encode(src_ids.unsqueeze(0))
        token, words, attn = torch.tensor([[SOS]]), [], []
        for _ in range(max_len):
            out = model.decode(token, enc_out, state, pad_mask)
            logits, state, weights = out
            next_id = logits[0, -1].argmax().item()
            attn.append(weights[0, -1])              # 이 단어를 만들 때의 분포
            if next_id == EOS:
                break
            words.append(tar_words[next_id])
```

```
            token = torch.tensor([[next_id]])
    return words, torch.stack(attn)

Xt, _, Yt = data["test"]
print("\n테스트 데이터의 문장 (처음 보는 문장)")
for i in [0, 1, 2, 3, 4, 5]:
    source = " ".join(src_words[k] for k in Xt[i].tolist() if k)
    answer = " ".join(tar_words[k] for k in Yt[i].tolist() if k and k ≠ EOS)
    print(f"  입력: {source:26s} 정답: {answer:30s}",
          f"번역: {' '.join(translate(Xt[i])[0])}")

refs = [[tar_words[k] for k in y.tolist() if k and k ≠ EOS] for y in Yt[:500]]
hyps = [translate(x)[0] for x in Xt[:500]]
exact = sum(r == h for r, h in zip(refs, hyps))
bleu = corpus_bleu([[r] for r in refs], hyps)
print(f"테스트 500문장: 정답과 완전히 같은 번역 {exact}개, BLEU {bleu:.4f}")

# 번역할 때 영어의 어느 단어를 봤는가
src_ids = Xt[5]
words, attn = translate(src_ids)
source = [src_words[k] for k in src_ids.tolist() if k]
print("\n어텐션 분포 (행 = 만든 단어, 열 = 영어 단어)")
print("          " + "".join(f"{w:>9}" for w in source))
for w, row in zip(words + ["<eos>"], attn):
    print(f"{w:>10}" + "".join(f"{v:9.2f}" for v in row[:len(source)].tolist
()))
torch.save({"history": history, "refs": refs, "hyps": hyps, "source": source,
            "words": words + ["<eos>"], "attn": attn[:, :len(source)]},
           "attention_result.pt")
```

실행 결과

```
에포크  1: 학습 손실 4.1432 | 검증 손실 3.1519, 정확도 0.5114  ← 저장
에포크  2: 학습 손실 2.7284 | 검증 손실 2.4748, 정확도 0.5905  ← 저장
에포크  3: 학습 손실 2.1138 | 검증 손실 2.1124, 정확도 0.6333  ← 저장
에포크  4: 학습 손실 1.6813 | 검증 손실 1.8738, 정확도 0.6638  ← 저장
에포크  5: 학습 손실 1.3368 | 검증 손실 1.7185, 정확도 0.6853  ← 저장
에포크  6: 학습 손실 1.0552 | 검증 손실 1.6006, 정확도 0.7043  ← 저장
에포크  7: 학습 손실 0.8304 | 검증 손실 1.5232, 정확도 0.7168  ← 저장
에포크  8: 학습 손실 0.6547 | 검증 손실 1.4844, 정확도 0.7207  ← 저장
에포크  9: 학습 손실 0.5238 | 검증 손실 1.4507, 정확도 0.7295  ← 저장
에포크 10: 학습 손실 0.4279 | 검증 손실 1.4522, 정확도 0.7288

테스트 데이터의 문장 (처음 보는 문장)
  입력: they all did it .          정답: ils le firent tous .          번역:
ils l ont tous fait .
  입력: we all hate tom .          정답: nous detestons tous tom .     번역:
nous aimons tous tom .
  입력: that seemed to help .      정답: ca a paru aider .             번역:
cela aide nous aider .
  입력: this gold is mine .        정답: cet or est mien .             번역: c
est a moi qui est moi .
  입력: the door creaked .         정답: la porte grinca .             번역:
la porte s est ouverte .
  입력: get out of the truck .     정답: sortez du camion !            번역:
sors de la camionnette !
테스트 500문장: 정답과 완전히 같은 번역 51개, BLEU 0.3007

어텐션 분포 (행 = 만든 단어, 열 = 영어 단어)
                get      out       of      the    truck        .
      sors     0.00     0.00     0.00     0.00     0.00     1.00
        de     0.00     0.00     0.00     0.00     0.00     1.00
```

```
          la     0.00     0.00     0.00     0.00     0.00     1.00
camionnette      0.00     0.00     0.00     0.00     0.00     1.00
           !     0.00     0.00     0.00     0.00     0.00     1.00
       <eos>     0.00     0.00     0.00     0.00     0.00     1.00
```

직접 해 보기 `decode`에서 패딩을 가리는 두 줄(`masked_fill`)을 지우고 다시 학습하면 결과가 어떻게 달라질까요? 어텐션 분포에서 패딩 자리의 값은 어떻게 되나요?

확인 문제

17-03 확인 문제 한 명씩 돌아가며 풀고, 푼 사람 이름을 적어 주세요

1 어텐션을 쓰는 번역기는 인코더의 모든 시점의 출력을 디코더에 넘긴다. (O / X) 이름 _____

2 어텐션 지도를 보면 단어를 만들 때 입력의 어디를 봤는지 알 수 있다. (O / X) 이름 _____

3 스코어 행렬의 모양 (B, T, L)에서 L은? 이름 _____

① 입력 문장의 길이 ② 출력 문장의 길이 ③ 은닉 크기

4 어텐션에서 패딩 자리를 가리는 방법은? 이름 _____

① 스코어에 아주 작은 값을 넣는다 ② 패딩을 지운다 ③ 손실에서만 뺀다

5 이 실습에서 쓴 스코어 함수는 두 은닉 상태의 _____이다. 이름 _____

6 어텐션 값과 디코더의 은닉 상태를 이어 붙인 뒤 가중치 W_c를 곱하고 _____ 함수를 씌운다. 이름 _____

7 16장의 번역기와 비교해 코드에서 무엇이 달라졌고 결과가 어떻게 달라졌는지 설명해 보세요. 이름 _____

원문: 17-03 어텐션 메커니즘을 이용한 번역기 구현 — https://wikidocs.net/216495 (교재는 어텐션을 한 번만 계산하는 간단한 구현입니다. 이 자료는 시점마다 계산하고 패딩을 가리도록 다시 구현했고, 30 에포크를 10 에포크로 줄였습니다. 16장 모델과의 비교와 어텐션 지도는 이 자료에서 추가했습니다)

오늘의 요약

다시 보기



정의 모음

개념	정의	예시
어텐션	매 시점 입력 전체를 다시 보고, 필요한 곳에 집중	étudiant ← student
Query / Key, Value	디코더의 은닉 상태 / 인코더의 은닉 상태들	s_t / $h_1, \ldots, h_N$
어텐션 스코어	Query와 Key의 비슷한 정도	[0.70, 1.00, 0.35, 2.50]
어텐션 분포	스코어에 소프트맥스를 씌운 비율 (합 1)	[0.11, 0.15, 0.08, 0.66]
어텐션 값	그 비율로 Value를 가중합한 벡터	[0.15, 1.48, 0.81]
실습 결과	같은 데이터에서 어텐션을 더한 효과	BLEU 0.2471 → 0.3007
루옹 / 바다나우	내적, LSTM 뒤 / 작은 신경망, LSTM 앞	$s_t^T h_i$ / $W_a^T \tanh(\cdot)$

다음

다음 자료: 18장 트랜스포머 (10/15)

오늘은 RNN으로 만든 seq2seq에 어텐션을 **덧붙였습니다.** 다음에는 RNN을 아예 빼고 어텐션만으로 인코더와 디코더를 만듭니다. 오늘의 Query, Key, Value와 스코어 → 분포 → 가중합이 그대로 쓰이고, 디코더가 입력을 보는 어텐션에 더해 문장이 자기 자신을 보는 셀프 어텐션이 나옵니다.

정답과 해설

17-01 어텐션 메커니즘

1. X — 인코더의 모든 시점의 은닉 상태를 씁니다.
2. O — 소프트맥스를 씌웠기 때문입니다.
3. ②
4. ①
5. 소프트맥스
6. 가중
7. 예시 답: 지금 하려는 말(Query)을 떠올리고 원문의 단어마다 얼마나 관련 있는지 가늠하는 것이 스코어, 그것을 합이 1인 비율로 바꾼 것이 눈이 머무는 시간의 비율인 분포, 그 비율대로 원문 단어들의 뜻을 섞어 떠올린 것이 어텐션 값이다.

17-02 바다나우 어텐션

1. O — W_a, W_b, W_c
2. X — 스코어를 구하는 방법만 다르고, 소프트맥스와 가중합은 같습니다.
3. ②
4. ①
5. 루옹
6. tanh (하이퍼볼릭 탄젠트)
7. 예시 답: 같은 점은 스코어 → 소프트맥스 → 가중합의 순서로 어텐션 값을 구한다는 것. 다른 점은 스코어를 내적으로 구하느냐 작은 신경망으로 구하느냐(또는 어텐션 값을 LSTM 뒤에 쓰느냐 앞에 쓰느냐).

17-03 어텐션 메커니즘을 이용한 번역기 구현

1. O
2. O
3. ①
4. ①
5. 내적
6. tanh
7. 예시 답: 인코더가 모든 시점의 출력을 넘기고, 디코더가 LSTM 뒤에서 스코어 → 분포 → 가중합으로 어텐션 값을 구해 은닉 상태와 이어 붙인 뒤 출력층에 넣도록 바뀌었다. BLEU가 0.2471에서 0.3007로, 완전히 같은 번역이 34개에서 51개로 늘었다. 다만 어텐션은 기대와 달리 마지막 토큰에만 쏠렸다.

용어 정리

용어	정의
어텐션	출력 단어를 예측할 때마다 입력 전체를 다시 참고하되, 관련 있는 부분에 집중하는 방법
쿼리 / 키 / 값	질문하는 쪽 / 질문과 맞춰 볼 대상 / 가져올 내용
어텐션 스코어	쿼리와 키가 비슷한 정도를 나타내는 점수
어텐션 분포, 어텐션 가중치	스코어에 소프트맥스를 씌운 합 1의 비율 / 그 각각의 값
어텐션 값	어텐션 분포를 비율로 값들을 가중합한 벡터. 시점마다의 컨텍스트 벡터
닷-프로덕트(루옹) 어텐션	스코어를 내적으로 구하는 어텐션
바다나우 어텐션	스코어를 작은 신경망으로 구하고, 어텐션 값을 LSTM의 입력에 넣는 어텐션
어텐션 지도	출력 단어마다의 어텐션 분포를 표로 그린 것